

Reviewer Pack — Minimal Order of Automorphism Groups and Communication Compl...

Entry dac862ed5e5c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 07:50:46 UTC

Minimal Order of Automorphism Groups and Communication Complexity

Entry ID: dac862ed5e5c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 07:50:46 UTC

1. Conjecture

Field A (mathematical branch): Group Theory (specifically, automorphism groups of finite groups)

Field B (complexity object): Communication Complexity

Statement:

For any given Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the minimal order of the automorphism group of its associated permutation representation is linearly correlated with its communication complexity $c(f)$, such that $|\text{Aut}(G_f)| = \Theta(c(f))$, where G_f is the permutation group representing the action of f on the Boolean cube.

Rationale (proposer's reasoning):

Automorphism groups capture symmetry properties of Boolean functions, and it is plausible that their order can reflect the complexity of communicating information about these functions. If communication complexity corresponds to a kind of 'information symmetry', then a higher order of automorphisms might correspond to more complex communication tasks.

Taxonomy category: group_theory_x_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a940ce6fbb4a9ef6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the minimal order of the automorphism group and communication complexity for at least 80% of the random Boolean functions ($n \leq 40$) is ≥ 0.7 , with a $p\text{-value} \leq 0.05$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.7s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def permutation_representation(f):
        n = len(f)
        G_f = []
        for i in range(2**n):
            permuted = [f[i ^ (1 << j)] for j in range(n)]
            G_f.append(permuted)
        return G_f

    def automorphism_group(G_f):
        n = int(math.log2(len(G_f)))
        aut_group = []
        for i in range(1, 2**n):
            if all(G_f[i ^ (1 << j)] == G_f[j] for j in range(n)):
                aut_group.append(i)
        return aut_group

    def communication_complexity(f):
        n = len(f)
        # Simplified example: Hamming distance
        return sum(1 for i in range(2**n) if f[i] != f[i ^ 1])

    results = []
    for _ in range(30): # Ensure at least 30 instances per seed
        n = random.choice([5, 10, 15, 20, 30, 40])
        f = generate_boolean_function(n)
        G_f = permutation_representation(f)
        aut_group = automorphism_group(G_f)
```

```

        c_f = communication_complexity(f)
        results.append((n, len(auto_group), c_f))

n_max = max(n for _, _, _ in results)
if n_max < 16:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "n_max < 16"
    }

C = [c for _, _, c in results]
A = [a for _, a, _ in results]
mean_C = sum(C) / len(C)
mean_A = sum(A) / len(A)
numerator = sum((C[i] - mean_C) * (A[i] - mean_A) for i in range(len(C)))
denominator = math.sqrt(sum((C[i] - mean_C)**2 for i in range(len(C))) * sum((A[i] - mean_A)**2 for i in range(len(C))))
if denominator == 0:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "denominator is zero"
    }
pearson_corr = numerator / denominator

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": pearson_corr,
    "instances_tested": len(results),
    "n_max": n_max,
    "conjecture_holds": pearson_corr >= 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{trial_result}...}}")
        results.append(trial_result)

    mean_corr = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_corr} std=None support_fraction=1.0")
    elif support_fraction >= 0.8:

```

```

print(f"RESULT: SUPPORTED mean={mean_corr} std=None support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_ho"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3c68e591.py", line 97, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3c68e591.py", line 49, in run_trial
    G_f = permutation_representation(f)
          ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3c68e591.py", line 28, in permutation_r
    permuted = [f[i ^ (1 << j)] for j in range(n)]
                ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means that the Pearson correlation coefficient could not be calculated. | next: Re-run the test to ensure it completes successfully and produces the required data for calculating the Pearson correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14159
2	preregistration	ollama_remote	glm4:latest	0	0	12003
3	novelty	ollama_remote	glm4:latest	0	0	8223
4	novelty	ollama_remote	glm4:latest	0	0	8487
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16321
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12524
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	46609
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17868
9	judge	ollama_remote	glm4:latest	0	0	17075

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 153269 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/dac862ed5e5c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/dac862ed5e5c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/dac862ed5e5c.tar.gz (if generated)